

INFORME DE ENTREGA: APLICACIÓN DE CONSOLA Y BASE DE DATOS

Proyecto: Sistema de Telemetría y Registro de Pruebas del Auto

Fecha de Entrega: 22/09/2026

```
import sqlite3

# Conexión a la base de datos del sistema del auto
conn = sqlite3.connect("registro_auto.db")
cursor = conn.cursor()

# Creación de la tabla si no existe (almacena registros de velocidad, giros y sensores del auto)
cursor.execute("""
CREATE TABLE IF NOT EXISTS pruebas_auto (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    conductor TEXT NOT NULL,
    velocidad_max REAL NOT NULL,
    vueltas_completadas INTEGER NOT NULL,
    tiempo_segundos REAL NOT NULL,
    estado_sensores TEXT NOT NULL
)
""")
conn.commit()

def registrar_prueba():
    """Registra una nueva prueba de manejo/autonomía realizada con el auto."""
    print("\n--- REGISTRAR NUEVA PRUEBA DEL AUTO ---")
    conductor = input("Nombre del conductor / operador: ")
    velocidad_max = float(input("Velocidad máxima alcanzada (km/h): "))
    vueltas = int(input("Vueltas / circuitos completados: "))
    tiempo = float(input("Tiempo total de la prueba (segundos): "))
    estado_sensores = input("Estado del sistema de infrarrojo/frenado (OK / Falla / Calibrar): ")

    cursor.execute(
        "INSERT INTO pruebas_auto (conductor, velocidad_max, vueltas_completadas, tiempo_segundos, estado_sensores) "
        "VALUES (?, ?, ?, ?, ?)",
        (conductor, velocidad_max, vueltas, tiempo, estado_sensores)
    )
    conn.commit()
    print("✓ Prueba del auto registrada correctamente.")

def mostrar_pruebas():
    """Muestra el historial completo de telemetría y pruebas del auto."""
    print("\n--- HISTORIAL DE PRUEBAS DEL AUTO ---")
```

```

cursor.execute("SELECT * FROM pruebas_auto")
pruebas = cursor.fetchall()

if pruebas:
    for p in pruebas:
        print(f"ID: {p[0]} | Conductor: {p[1]} | Vel. Máx: {p[2]} km/h | Vueltas: {p[3]} | Tiempo: {p[4]}s | Estado: {p[5]}")
    else:
        print("No hay registros de pruebas en la base de datos.")

def buscar_prueba_conductor():
    """Busca registros de pruebas filtrando por el nombre del conductor u operador."""
    print("\n--- BUSCAR PRUEBAS POR CONDUCTOR ---")
    nombre = input("Nombre a buscar: ")
    cursor.execute("SELECT * FROM pruebas_auto WHERE conductor LIKE ?", ('%' + nombre + '%',))
    resultados = cursor.fetchall()

    if resultados:
        for p in resultados:
            print(f"ID: {p[0]} | Conductor: {p[1]} | Vel. Máx: {p[2]} km/h | Vueltas: {p[3]} | Tiempo: {p[4]}s | Estado: {p[5]}")
        else:
            print("No se encontraron registros para ese conductor.")

def modificar_prueba():
    """Modifica los datos de un registro de prueba del auto por su ID."""
    print("\n--- MODIFICAR REGISTRO DE PRUEBA ---")
    try:
        id_prueba = int(input("ID de la prueba a modificar: "))
        nuevo_conductor = input("Nuevo conductor / operador: ")
        nueva_velocidad = float(input("Nueva velocidad máxima (km/h): "))
        nuevas_vueltas = int(input("Nuevas vueltas completadas: "))
        nuevo_tiempo = float(input("Nuevo tiempo total (segundos): "))
        nuevo_estado = input("Nuevo estado de sensores (OK / Falla / Calibrar): ")

        cursor.execute("""
            UPDATE pruebas_auto
            SET conductor = ?, velocidad_max = ?, vueltas_completadas = ?, tiempo_segundos = ?,
            estado_sensores = ?
            WHERE id = ?
            """, (nuevo_conductor, nueva_velocidad, nuevas_vueltas, nuevo_tiempo, nuevo_estado,
            id_prueba))

        if cursor.rowcount > 0:
            conn.commit()
            print("Registro de prueba modificado correctamente.")
        else:
            print("No se encontró ningún registro con ese ID.")
    except ValueError:
        print("Error: Ingrese valores numéricos válidos en velocidad, vueltas o tiempo.")

def eliminar_prueba():
    """Elimina un registro de prueba del auto por su ID."""
    print("\n--- ELIMINAR REGISTRO DE PRUEBA ---")
    try:

```

```

id_prueba = int(input("ID de la prueba a eliminar: "))
cursor.execute("DELETE FROM pruebas_auto WHERE id = ?", (id_prueba,))

if cursor.rowcount > 0:
    conn.commit()
    print(" Registro eliminado correctamente.")
else:
    print("No se encontró ningún registro con ese ID.")
except ValueError:
    print("Error: Ingrese un ID numérico válido.")

def menu():
    """Menú principal interactivo de la consola del auto."""
    while True:
        print("\n=====")
        print(" SISTEMA DE CONTROL Y TELEMETRÍA - AUTO IR ")
        print("=====")
        print("1. Registrar nueva prueba del auto")
        print("2. Mostrar historial de pruebas")
        print("3. Buscar pruebas por conductor")
        print("4. Modificar registro de prueba")
        print("5. Eliminar registro de prueba")
        print("0. Salir")

        opcion = input("Elegí una opción: ")

        match opcion:
            case "1":
                registrar_prueba()
            case "2":
                mostrar_pruebas()
            case "3":
                buscar_prueba_conductor()
            case "4":
                modificar_prueba()
            case "5":
                eliminar_prueba()
            case "0":
                print("¡Cerrando sistema de telemetría del auto!")
                break
            case _:
                print("Opción inválida. Intente nuevamente.")

# Ejecución de la aplicación
menu()
conn.close()

```

2. Bitácora de Pruebas, Problemas y Soluciones

- **Problema 1: `sqlite3.OperationalError: no such table: pruebas_auto`**
 - **Descripción:** Al iniciar la consola por primera vez en un entorno limpio, el programa arrojaba un error porque el archivo `registro_auto.db` no contaba con la estructura de tablas necesaria.
 - **Solución propuesta:** Se agregó la sentencia `CREATE TABLE IF NOT EXISTS` durante la inicialización de la conexión en Python para que la tabla se cree de forma automática en la primera ejecución sin intervención externa.
- **Problema 2: Cierre inesperado por tipo de dato incorrecto (`ValueError`)**
 - **Descripción:** Al intentar ingresar letras o símbolos en los campos que requieren enteros o flotantes (velocidad, tiempo o vueltas), el script colapsaba deteniendo la ejecución de la consola.
 - **Solución propuesta:** Se estructuró la captura de entradas mediante bloques `try/except ValueError` en las funciones de edición y eliminación, informando al usuario del error sin interrumpir el flujo del programa.
- **Problema 3: Modificación o eliminación silenciosa ante IDs inexistentes**
 - **Descripción:** Si el usuario ingresaba un número de ID que no existía en la base de datos, el programa notificaba que la acción había sido exitosa aunque ningún registro hubiese sido afectado.
 - **Solución propuesta:** Se incorporó la validación mediante `cursor.rowcount` para comprobar si la consulta SQL afectó efectivamente al menos una fila antes de imprimir el mensaje de confirmación.

3. Modo de Uso y Funcionamiento de la Aplicación

La aplicación funciona en su totalidad a través de la **consola / terminal** de comandos. No requiere la instalación de gestores visuales externos ni la creación manual de bases de datos.

1. **Gestión Automática:** Al ejecutar `python app.consola.py`, el script crea el archivo `registro_auto.db` y administra la tabla de manera directa.
2. **Navegación:** Toda la lectura, ingreso, edición y borrado de registros se realiza mediante el menú numérico interactivo directamente en la consola.